
Psec Research Project

Reversing Obfuscated Code Using LLMs

Valentin Hego, Samuel Cattelier, Mosè Arcaro, Valentin Boue, Natale Oricelli

M1 Cybersécurité SML EIT

2026



Abstract

Code obfuscation is widely used to protect intellectual property and harden software against reverse engineering since it imposes a significant cost increase on legitimate (and also malicious) analysts: manual deobfuscation is slow, error-prone, and requires deep level of expertise. This report evaluates whether modern Large Language Models (LLMs) can accelerate the deobfuscation workflow. Using Gemini 3.1 Pro, we designed a controlled experiment in which a large set of C programs across three patterns (arithmetic operations, loops, and function calls) were obfuscated using three distinct tools (Tigress, MoVfuscator, and Kovid), decompiled, and submitted to the model for reconstruction. To avoid subjective bias, outputs were also scored objectively using CodeBERTScore, with an empirically calibrated success threshold of $F_1 = 0.70$. Results show that LLMs recover the semantic intent of obfuscated code with surprisingly high fidelity, with scores typically between 0.76 and 0.92, and Tigress proving the most resistant obfuscator. A real-world case study on a stripped, heavily obfuscated Go binary further confirmed that LLM assistance substantially reduced the time required to identify hidden command-and-control behaviour. We conclude that LLMs are powerful accelerators for reverse engineering, though expert supervision remains necessary to handle hallucinations and highly complex transformations.

◆ Contents

1	Introduction	3
2	Methodology	3
2.1	Experimental Setup and Automation	3
2.2	Obfuscation Tools	4
2.2.1	Tigress	4
2.2.2	MoVfuscator	5
2.2.3	Kovid	5
2.2.4	CObfuscator (discarded)	5
2.3	Reverse Engineering Workflow	6
2.4	Evaluation Metric: CodeBERTScore	6
3	Results	7
3.1	Quantitative Results by Tool	8
3.2	Real-World Case Study	9
3.3	Exploratory Work: Ghidra MCP with Claude	9
4	Discussion	10
5	Conclusion	11
6	Future Work	12

◆ 1 Introduction

Obfuscation is the practice of making computer code intentionally difficult to understand while preserving its original behaviour. It is widely and used by routine to protect intellectual property, harden software against tampering, and, in adversarial contexts, to hide the logic of malicious binaries. Whatever the intent, obfuscation creates a “barrier of complexity” that the analyst must overcome. Manual deobfuscation is highly skilled work: it is slow, error-prone, and tightly coupled to the analyst’s familiarity with both the obfuscation technique and the underlying architecture.

This report addresses a central research question: *Can emerging Large Language Models (LLMs) effectively accelerate and simplify the deobfuscation workflow?* To answer it, we built a reproducible experimental pipeline and submitted controlled samples to Gemini 3.1 Pro for reconstruction. Three obfuscators were used to produce stylistically distinct outputs, three categories of program structure were tested (arithmetic operations, loops, and function calls), and complexity was increased progressively across a large number of samples per pattern.

Evaluating the quality of LLM output presents its own difficulty. Initial evaluation was conducted by human inspection, but the subjectivity of manual scoring, combined with the variable expertise of individual reviewers, motivated a further investigation toward CodeBERTScore, an automated, semantic similarity metric purpose-built for source code generated by AI. We complement our benchmark with a real-world case study on a stripped, obfuscated Go binary to evaluate the practical usefulness of LLM assistance outside the laboratory.

The remainder of the report is organised as follows. Section 2 details the experimental methodology, including tooling, the obfuscators used, and the evaluation metric. Section 3 presents the quantitative results, the real-world case study, and an exploratory experiment using Ghidra MCP with Claude. Section 4 discusses what the results mean and where they fall short. Sections 5 and 6 conclude with our findings and directions for future work.

◆ 2 Methodology

■ 2.1 Experimental Setup and Automation

The experimental pipeline was designed around three principles: reproducibility, comparability, and progressive difficulty. All source programs were written in C and organised into three structural categories (arithmetic operations, loops, and function calls), with many samples per category. Complexity was constantly increased within each category (e.g. loop1 is simpler than loop2, and so on), yielding a dataset designed to expose the point at which LLM reconstruction begins to degrade.

To guarantee reproducibility, the generation of experimental files was fully automated through a Continuous Integration (CI) pipeline. A `.gitlab-ci.yml` file launches a Docker-hosted job on every push to the GitLab repository. The job compiles the C source, applies the requested obfuscation pass with the configured parameters, and emits the binary into an outputs folder

retained as GitLab artifacts. A fixed random seed (seed = 0) is supplied to every obfuscation tool. Many obfuscators introduce randomness during their transformations, which would otherwise make outputs non-deterministic; fixing the seed guarantees that a given input and configuration always produces the same binary, which is essential for valid comparison across runs.

Ghidra was used as the disassembler and decompiler for all manual and scripted analysis. Custom scripts exported decompiled functions to external files, which were then submitted to the LLM. We also explored an alternative input modality based on `objdump -d` only output: the raw disassembly of the binary, provided directly to the LLM without a graphical decompiler in the loop. This was tested as a minor experiment to verify whether reconstruction is feasible from machine instructions alone, and our observations indicate that it is.

■ 2.2 Obfuscation Tools

Three obfuscators were selected for the final experiments: Tigress, MoVfuscator, and Kovid. Two further tools, OLLVM and Obfusc8, were evaluated and discarded. OLLVM is no longer maintained, and Obfusc8 is Windows-only. A GitHub Actions runner was briefly tried to host Obfusc8 but proved too slow for the volume of experiments required. A fourth tool, CObfuscator, was discarded after some testing exposed it as unreliable.

▲ 2.2.1 Tigress

Tigress is a source-to-source C/C++ obfuscator: it transforms an input `.c` file into a semantically equivalent but significantly harder-to-read `.c` file. Its defining feature among the tools we tested is that it exposes multiple, composable transformation passes, the output of one pass becoming the input of the next. This composability creates compatibility constraints: certain orderings produce invalid output. For example, applying Virtualize before Split causes errors because the virtualizer generates internal structures that the splitter cannot safely decompose. Where both passes were used, we consistently placed Split before Virtualize.

The Tigress transformations used in this study were:

- **Flatten** (Control Flow Flattening): replaces the original control flow graph with a single, flat `switch` statement driven by a state variable. Each original basic block becomes a case in the switch; the state variable selects the next block. The naturally readable execution path is hidden behind an opaque dispatcher loop, making the original CFG a way to harder to reconstruct.
- **Split** (Function Splitting): artificially chops a single function into several smaller, interconnected sub-functions, disrupting modularity and making it tedious to follow logic across boundaries.
- **Virtualize**: converts the original code into a custom, randomised bytecode and generates a unique virtual machine to interpret it, hiding the original instructions from static analysis tools entirely.

- **EncodeArithmetic** (Mixed Boolean-Arithmetic): replaces simple arithmetic operations with complex, mathematically equivalent bitwise formulas, making calculations incomprehensible at first impact.
- **EncodeLiterals**: removes hardcoded constants such as strings and integers, replacing them with code that computes their values dynamically at the exact moment they are needed.

▲ 2.2.2 MoVfuscator

MoVfuscator is a C compiler that transforms source code into a binary composed almost exclusively of `mov` instructions. The resulting executable is significantly larger than the original and contains virtually no conventional control-flow instructions: no conditional jumps, no arithmetic operations. Internally, the compilation pipeline first lowers C into a Brainfuck-like intermediate code representation, then into `mov`-only assembly. All computation, both arithmetic and control flow, must be emulated through memory transfers and data movement.

To simulate conditional branching without branch instructions, MoVfuscator evaluates both execution paths simultaneously and selects the correct result indirectly through data manipulation, while the alternative path operates on dummy values that never reach the final program state. This removes explicit control-flow patterns from the binary and severely complicates static analysis: common patterns such as loops, arithmetic expressions, and conditional statements become difficult to identify because of the uniformity of the instruction set and the heavy reliance on memory indirection.

▲ 2.2.3 Kovid

Kovid is a lighter-weight obfuscator built as a collection of LLVM and GCC plugins. Unlike Tigress, it takes source code in and emits an obfuscated executable directly. Its main transformations are: function renaming; insertion of non-functional “dummy” instructions; removal of debug metadata and unused code; arithmetic instruction pattern transformation; string encryption of plaintext literals in the binary; and a combined control-flow taint pass that mixes opaque predicates, control flow flattening with dispatcher blocks, and state variable injection.

▲ 2.2.4 CObfuscator (discarded)

CObfuscator is a Python utility intended to transform readable C into a functionally equivalent but obfuscated form. Our evaluation uncovered several systemic defects that broke functional equivalence and syntactic validity:

- **Syntax errors in operators.** The tool inserts whitespace into compound operators (e.g. transforming `<=` into `< =`), producing invalid C that fails to compile.
- **String literal corruption.** Obfuscated identifiers are injected into `printf` format strings.

For example, `printf("%d\n")` may be corrupted into `printf("%d\_variableName")`, altering the intended output and potentially causing runtime errors.

- **Structural malformation of loops.** The transformation logic frequently replaces the increment expression of a for loop with the loop body, producing structurally invalid statements that violate the C standard.

Because the tool commonly failed to produce compilable, functionally equivalent output, it was labeled as unreliable and removed from the workflow.

■ 2.3 Reverse Engineering Workflow

Each obfuscated binary was first manually reverse-engineered using Ghidra to establish a ground-truth reference. The same decompiled output was then submitted to the LLM with a deliberately minimal prompt:

"Ghidra gives me this, can you give me your best guess of the real program?"

No additional context was provided: no hints about the obfuscation technique, no indication of the original program's purpose. This was a deliberate methodological choice: the goal was to evaluate the LLM as a black box, isolating its raw reconstruction capability from any context-priming effect. We are aware that supplying additional context (e.g. naming the obfuscator, or describing the expected program structure) is known to improve LLM output, and this is flagged as a possible path for future work.

We also evaluated GhidrAssist, a Ghidra plugin that uses an LLM to explain code and rename functions and variables. While GhidrAssist is useful for general reverse engineering, particularly for annotating identifiers and producing call graphs, it does not perform decompilation or attempt to produce an estimate of the original C source. For our specific use case of reconstructing original code from an obfuscated binary, it could not contribute directly.

■ 2.4 Evaluation Metric: CodeBERTScore

Initial evaluations relied on human inspection of the LLM's output. This proved unsatisfactory: judgments varied between reviewers depending on their personal familiarity with the obfuscation technique, and there was no principled way to score partial reconstructions. To eliminate this subjectivity we adopted CodeBERTScore [1] as an automated, objective metric.

Before settling on CodeBERTScore, we briefly considered CodeBLEU [2] as a candidate metric. We ruled it out because its scoring is dominated by token-surface similarity: it captures syntactic differences between two snippets rather than what those snippets mean. For our use case, where the same logic may legitimately be expressed with very different identifiers, control structures, or statement orderings, a semantic metric is essential.

CodeBERTScore is derived from BERTScore, originally developed for machine-translation evaluation, and is specifically engineered for AI-generated source code. Unlike surface-level

metrics such as BLEU, which rely on surface matching, CodeBERTScore evaluates the semantic equivalence of tokens within their structural context, using a pre-trained code-aware BERT model to encode both the reference and the candidate. Each token is converted into a contextualised vector embedding, and similarity is measured between embeddings rather than between raw text. This makes the metric robust to superficial differences (variable renaming, whitespace, equivalent syntactic restructurings) that should not count as failures of reconstruction. It is also fully automated, which sidesteps the need for hand-authored test cases.

The scoring pipeline operates in the following stages:

1. **Tokenization.** The reference and candidate code are decomposed into structural tokens.
2. **Encoding.** Tokens are passed through the CodeBERT neural network to produce contextualised vector embeddings.
3. **Masking.** Non-essential syntactic tokens (parentheses, braces, language-specific punctuation) are filtered out so the metric concentrates on semantic logic.
4. **Cosine similarity.** Vector distances between candidate and reference embeddings are computed to quantify semantic closeness.
5. **Precision and recall.** The similarity matrices are aggregated into a precision score (relevance of generated tokens) and a recall score (coverage of reference tokens).
6. **F-measure.** Precision and recall are combined into a harmonic mean. We use F_1 ; the original CodeBERTScore paper recommends F_3 for code generation, which weights recall more heavily, but we preferred an equally weighted score because our goal is to measure general utility for human analysts rather than to demand flawless, directly compilable output.

Because CodeBERTScore is a comparative ranking metric rather than an absolute grading system, it requires calibration before its scores can be interpreted as pass/fail. We ran two control experiments:

- Comparing two semantically equivalent but syntactically distinct implementations of the same algorithm yielded an F_1 of **0.81**.
- Comparing two entirely unrelated code snippets yielded an F_1 of **0.65**.

On the basis of these empirical baselines, we set $F_1 = 0.70$ as the functional success threshold for this study. Any LLM output scoring below this benchmark is categorised as a failure.

◆ 3 Results

■ 3.1 Quantitative Results by Tool

For each obfuscator and each program pattern, we computed the mean F_1 score between the clean original code and the LLM’s reconstruction across the full set of samples of progressively increasing complexity. The results are summarised in Tables 1, 2 and 3.

Pattern	Mean F_1
Loops	~ 0.83
Function Calls	~ 0.79
Arithmetic Operations	~ 0.76

Table 1: Tigress with all five passes combined.

Combining all five Tigress passes produces the hardest-to-reverse output in our experiment. Even so, the mean scores remain above the 0.70 threshold across all three patterns, though arithmetic, which is hit directly by EncodeArithmetic and indirectly by Virtualize, comes closest to the failure line.

Pass	Loops	Function Calls	Arithmetic
Virtualize	~ 0.83	~ 0.84	~ 0.80
Split	~ 0.92	~ 0.90	~ 0.84
Flatten	~ 0.81	~ 0.83	~ 0.86
EncodeLiterals	~ 0.92	~ 0.90	~ 0.89
EncodeArithmetic	~ 0.91	~ 0.87	~ 0.86

Table 2: Tigress individual passes.

Taken single options, the picture is more faded. Most individual options leave the LLM with enough informations to reconstruct the original logic at $F_1 \geq 0.83$. Split, EncodeLiterals, and EncodeArithmetic are the least disruptive in isolation, producing scores at or above 0.84 across all patterns. Flatten and Virtualize are the harder individual options; in particular, Virtualize degrades arithmetic recovery the most among the individual options and Flatten produces the lowest score for loops.

Pattern	MoVfuscator	Kovid
Loops	~ 0.90	~ 0.80
Function Calls	~ 0.85	~ 0.80
Arithmetic Operations	~ 0.83	~ 0.70

Table 3: MoVfuscator and Kovid.

MoVfuscator’s all-mov transformation, despite its reputation for destroying high-level structure, did not prevent the LLM from recovering the original semantics: scores around 0.85 on average. Kovid sits at the boundary for arithmetic: at roughly 0.70 it is on the threshold. This is likely a consequence of Kovid’s aggressive removal of unused code: the tool tends to strip in-

intermediate computation and leave only the final outcome (for example, hardcoding the result of a calculation inside a final `printf`), which gives the LLM less material to work with.

Across all tools, the overall trend is what we expected: scores decline as program complexity increases. However, we observed several high-scoring outliers among some more complex programs, which we do not yet fully understand and which we flag as an interesting question for future work.

Beyond the numbers, the qualitative experience was striking: Gemini was able to identify most of the logic within seconds, a task that, working only from Ghidra's output, typically consumes far more time. This is especially valuable for newcomers, for whom Ghidra and the deobfuscation process can be overwhelming, particularly as program size grows.

■ 3.2 Real-World Case Study

To assess the practical usefulness of LLM-assisted deobfuscation outside of synthetic benchmarks, we analysed a real-world obfuscated Go binary. The sample was a stripped executable containing approximately 12,000 functions, protected with Go-specific obfuscation techniques. In particular, the `pcIntab` section, which is normally required by reverse engineering tools to recover function names and runtime metadata, had been deliberately altered, which prevented Ghidra's classical recovery routines from functioning and considerably complicated static analysis.

The LLM was widely used during the analysis: first to understand how the `pcIntab` structure had been modified, then to help design a fix that restored useful metadata. Several outdated Ghidra scripts written for older Go runtime versions also had to be updated, and the model accelerated this work by interpreting runtime structures and pointing out inconsistencies in the existing scripts.

This collaboration significantly reduced the time required to navigate the binary and understand its internal logic. The final analysis revealed hidden command-and-control (C2) behaviour embedded inside an otherwise legitimate-looking executable. Multiple stages of encrypted payload delivery were identified, along with several C2 communication endpoints. Expert supervision remained necessary throughout to validate the model's interpretations and confirm the final conclusions, but the overall throughput of the investigation was meaningfully higher than it would have been without LLM assistance.

■ 3.3 Exploratory Work: Ghidra MCP with Claude

In addition to the core experiments, we explored an alternative interaction model based on Ghidra MCP [9], a plugin that connects Ghidra to an LLM through the Model Context Protocol (MCP), a standardised communication protocol developed by Anthropic. Rather than manually exporting decompiled code and pasting it into the LLM chat interface, Ghidra MCP allows the LLM to interact with Ghidra directly: it can query the disassembler, retrieve decompiled functions, rename variables and functions, and navigate the binary autonomously, without manual copy-paste.

Architecturally, Ghidra exposes a set of actions through a locally hosted MCP server, and Claude (via Claude Desktop) connects to it as an MCP client, calling the authorised actions as tools during a conversation. Conceptually this is a significant upgrade over our baseline workflow: rather than a one-shot prompt with a single Ghidra export, the model can reason across multiple functions and progressively build an understanding of the binary in dialogue with the analyst.

In practice, two issues limited its usefulness for this project:

- **Platform availability.** Claude Desktop, which is required to act as the MCP client, was at the time of our experiments officially supported only on Windows and macOS. Our CI pipeline and most of our devices run on Linux. A community repackaging script (producing a .deb) was found on GitHub, but the resulting MCP server connection failed consistently in our environment. We eventually managed a complete working setup, and the experience was genuinely promising: Claude was able to navigate the binary, identify functions, and produce meaningful deobfuscation results.
- **Token-window saturation.** Obfuscated binaries, particularly those produced by Tigress with virtualization or by MoVfuscator, generate extremely verbose decompiled output. The model’s context window filled up rapidly, forcing the session to either truncate the analysis or restart and lose previously established context. For large or heavily obfuscated programs this made the workflow unreliable as a complete-analysis tool.

These setbacks slowed the project, and we ultimately chose a more practical solution. Gemini 3.1 Pro was selected as the primary LLM for the systematic experiments: it was available free of charge to students through Google’s academic programme, which mattered for a project requiring repeated API calls across a large number of samples, and its larger context window better accommodated the verbose decompiled output of moderately obfuscated binaries. Ghidra MCP remains an interesting direction for future work as Claude Desktop becomes more accessible across platforms and as context windows continue to grow.

◆ 4 Discussion

Two findings from this study are worth emphasising. First, across every obfuscator and every program pattern in our dataset, the mean F_1 score between the original C and the LLM’s reconstruction sits at or above the 0.70 success threshold we calibrated empirically. This is a strong result: it means that an off-the-shelf, general-purpose LLM, prompted as a black box without any hints about the obfuscation technique used, consistently recovers something semantically close to the original source. For an analyst staring at a screen of Ghidra output, this kind of approximate but semantically grounded reconstruction is often the difference between a productive afternoon and a wasted one.

Second, the differences between tools matter. Tigress proved the most resistant in our experiments, especially when its options were combined; this is consistent with its design as a

research-grade obfuscator with deliberate compositional complexity. MoVfuscator, despite the radical appearance of an all-mov binary, was relatively easy for the LLM to unwind, which suggests that the model is able to recognise and reason about MoVfuscator’s distinctive patterns even without being told what tool produced the binary. Kovid also sat at the easier end of the spectrum, with arithmetic recovery on the threshold. This is likely because Kovid’s aggressive dead-code elimination leaves the LLM with less material to work with rather than because the transformation itself is particularly hard to defeat.

Three caveats apply. The first concerns trust: LLMs sometimes hallucinate rather than admit uncertainty. A reconstructed function that looks plausible may not be faithful, and we observed instances where the model filled in confident-sounding code that did not match the original logic. Any production use of LLM assistance for reverse engineering must therefore include a validation step. The second is complexity: although the trend is gentle, error rates increase with program and obfuscation complexity, and our dataset does not extend into the regime of very large or production-grade obfuscated binaries. The third is methodological: we deliberately tested the LLM as a black box, with no hint about the obfuscation technique used. Supplying that context is known to improve results, and the scores reported here should therefore be read as a lower bound on what a context-aware workflow could achieve.

Finally, we noted several high-scoring outliers among complex programs: samples that should have been hard but were reconstructed unusually well. We do not yet have an explanation; this is one of the more interesting threads to follow in follow-up work.

◆ 5 Conclusion

Our research shows that Large Language Models are significant time-savers for reverse engineers working through the “barrier of complexity” that obfuscation imposes. On a controlled benchmark of many C programs obfuscated with three different tools, the LLM consistently produced reconstructions that exceeded our empirical threshold for semantic equivalence. In a real-world malware analysis scenario, the model contributed concretely by helping repair a mangled pcIntab section and identifying decryption routines, which materially accelerated the path to understanding the binary’s C2 workflow.

Three limitations qualify these findings:

- **Trust.** LLMs can hallucinate or “lie” instead of admitting they do not know. Independent validation of model output is non-negotiable.
- **Complexity.** Error rates rise with the density and complexity of both the program and the obfuscation applied to it.
- **Tool resistance.** Among the obfuscators we evaluated, Tigress, particularly when multiple passes are composed, was the most resistant to LLM analysis.

LLMs are powerful tools for reverse engineering, but they do not yet replace expert knowledge. The most productive workflow we observed combined both: the model handled the bulk

decompilation and structural reasoning, while the analyst validated, redirected, and resolved the cases where the model was unsure or incorrect.

◆ 6 Future Work

This study suggests several concrete directions:

- **Extend program complexity.** Our three-pattern dataset scales complexity within a controlled space. Extending the experiment to larger, more realistic codebases would produce more exhaustive results and probe the boundaries of LLM reconstruction.
- **Compare more LLMs and prompting strategies.** We evaluated a single model (Gemini 3.1 Pro) with a single minimal prompt. Comparing additional models, and varying the amount of context supplied to the model (naming the obfuscator, providing partial source, supplying documentation of the transformation), would establish how much of the performance ceiling can be recovered through prompt engineering.
- **Investigate high-scoring outliers in complex samples.** Several complex programs were reconstructed with unexpectedly high fidelity. Understanding why might reveal exploitable regularities in either the obfuscation tools or in how LLMs internalise code structure.
- **Revisit Ghidra MCP.** As Claude Desktop becomes more broadly available across platforms and as context windows continue to grow, the interactive MCP-based workflow may become viable for end-to-end analysis of large obfuscated binaries.

◆ References

- [1] S. Zhou, U. Alon, S. Agarwal, and G. Neubig. *CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code*. arXiv preprint arXiv:2302.05527, 2023. <https://arxiv.org/abs/2302.05527>
- [2] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma. *CodeBLEU: a Method for Automatic Evaluation of Code Synthesis*. arXiv preprint arXiv:2009.10297, 2020. <https://arxiv.org/abs/2009.10297>
- [3] Kovid Obfuscation Passes. <https://github.com/djolertrk/kovid-obfuscation-passes>
- [4] CObfuscator. <https://github.com/AleksaZatezalo/CObfuscator>
- [5] Tigress C Diversifier/Obfuscator. <https://tigress.wtf/>
- [6] The MoVfuscator: single instruction C compiler. <https://github.com/Battelle/movfuscator>
- [7] C. Domas. *The M/o/Vfuscator: Turning “mov” into a soul-crushing RE nightmare*. REcon 2015. <https://recon.cx/2015/slides/recon2015-14-christopher-domas-The-movfuscator.pdf>
- [8] GhidrAssist: LLM-assisted reverse engineering plugin for Ghidra. <https://github.com/symgraph/GhidrAssist>
- [9] Ghidra MCP: Model Context Protocol server for Ghidra. <https://github.com/lauriewired/ghidramcp>
- [10] Obfusc8. <https://github.com/x86byte/Obfusc8>
- [11] Ghidra. <https://ghidralite.com/>
- [12] Obfuscator-LLVM (OLLVM). <https://github.com/obfuscator-llvm/obfuscator>